■ Python Concurrency: Simple Tutorial

1■■ What Is Concurrency?

Concurrency means running multiple tasks so your program doesn't wait for one to finish before starting another.

Python gives us 3 tools:

• **Threads** → best for tasks that WAIT (I/O)

• **Processes** → best for CPU-heavy WORK

• **AsyncIO** → best for MANY waiting tasks (APIs, chatbots)

2■■ Threads: Best for I/O Tasks

Threads stay inside one Python program.

Example (Threads):

```
import threading

import time


def worker(name):

print(name, "starting")

time.sleep(1)

print(name, "finished")


t1 = threading.Thread(target=worker, args=("Thread-1",))

t2 = threading.Thread(target=worker, args=("Thread-2",))


t1.start()

t2.start()
```

```
t1.join()
t2.join()

print("All threads done!")
```

3■■ Processes: Best for CPU Tasks

Processes run in separate memory and use multiple CPU cores.

Example (Processes):

```
from multiprocessing import Process

def heavy_task():
total = 0
for i in range(1_000_000):
total += i
print("Done:", total)

if __name__ == "__main__":
p1 = Process(target=heavy_task)
p2 = Process(target=heavy_task)

p1.start()
p2.start()
p1.join()
p2.join()

print("All processes finished!")
```

4■■ AsyncIO: Best for Many Waiting Tasks

AsyncIO is great for chatbots and API calls.

Example (AsyncIO):

```
import asyncio

async def task(name, delay):
print(name, "started")
await asyncio.sleep(delay)
print(name, "finished")

async def main():
await asyncio.gather(
task("A", 2),
task("B", 1),
task("C", 3)
)

asyncio.run(main())
```

5■■ When to Use What?

Threads → I/O tasks

Processes → CPU-heavy tasks

AsyncIO → Many waiting tasks

6■■ Final Summary

Threads = same program, good for waiting tasks.

Processes = separate programs, good for heavy CPU work.

AsyncIO = one thread switching tasks very fast.